DEV JOURNEY

AGENT · FEATURE PLANNING COPILOT



A feature request is not an implementation plan.

Dev Journey Agent makes the decisions between the two visible, reviewable, and testable before code changes begin.

A portfolio case study about reducing the gap between what is asked and what a team understands to build.

THE PROBLEM

The gap appears before code.

A short request can sound clear while leaving the decisions that shape the solution unstated.

WHAT GETS ASKED

“Add a health-check endpoint.”

Enough to begin a conversation. Not enough to prove the right implementation.

WHAT STILL HAS TO BE DECIDED

- **Scope** — Which service and dependencies are included?
- **Contract** — What response defines healthy or unhealthy?
- **Ownership** — Who uses the signal and how is it monitored?
- **Proof** — Which tests demonstrate the desired behavior?

Unanswered decisions are where avoidable rework starts.

THE PROPOSAL

Make the thinking inspectable.

Dev Journey Agent turns an early request into a review package with assumptions, acceptance criteria, tests, and a visible decision state.

1

INPUT

Request + context

A ticket and read-only repository context start the planning process.

2

METHOD

Planning rules

Clarify gaps, define “done”, connect requirements to tests, then run a critic gate.

3

RESULT

Review package

A plan and evidence bundle, or a concise explanation of what must be clarified.

THE WORKFLOW

From request to evidence in six reviewable stages.

01

Read context

Retrieve repository structure and constraints.

02

Refine

Expose assumptions and missing details.

03

Define done

Set acceptance criteria and boundaries.

04

Specify + test

Describe behavior and test cases.

05

Critic gate

Check completeness before approval.

06

Export

Write the plan and traceable evidence.

Each stage leaves a checkpoint. The workflow can be resumed and reviewed without reconstructing the reasoning from memory.

THE DECISION RULE

Approve the plan only when the evidence supports it.

The same pipeline has two legitimate outcomes. It either produces a reviewable plan or states the question that blocks a reliable plan.



WHEN THE PLAN IS SUPPORTED

Approved

Export a spec, test-oriented cases, and status evidence for review.

WHEN A MATERIAL GAP REMAINS

Needs input

Stop and name the missing scope, data source, or decision instead of inventing the answer.

REVIEWABLE OUTPUTS

The result is a package people can inspect.

The goal is not a hidden recommendation. It is a practical handoff that connects planning decisions to the evidence behind them.

01

Feature specification

A clear request interpretation, requirements, acceptance criteria, and test intent.

02

Stakeholder case study

A readable explanation of the motivation, scope, outcome, and evidence.

03

Traceability bundle

Status JSON, checkpoints, telemetry, cost controls, and security audit evidence.

THE PRACTICAL BENEFIT

A reviewer can see what the agent concluded, why it concluded it, and what still needs a human decision.

Visible artifacts keep the claim inspectable.

VALIDATION

Two outcomes. Both are evidence.

The project was tested with a real read-only repository case and a credential-free ambiguity case. The point is to prove both progress and restraint.

APPROVED PATH

Health-check request

approved · exit code 0

Read-only repository context was retrieved and a complete feature plan with exports was generated.

SAFETY-GATED PATH

Ambiguous request

needs input · exit code 2

The pipeline stopped and named missing search scope and data source instead of approving an unreliable plan.

Release evidence: 83 tests · lint · type checks · HTML validation · secret audit

SCOPE

A smaller promise is a more credible one.

The portfolio project remains intentionally focused. Its value is planning clarity and inspectable evidence, not autonomous source-code changes or deployments.

IMPLEMENTED CORE

CLI planning pipeline

Manual GitHub Action

Provider-neutral local wrapper

Inspectable output and evidence

DELIBERATELY DEFERRED

Hosted service

Provider-specific chat adapters

Automatic issue and pull-request changes

Deployments

> **The deliverable is simple to evaluate: turn an early request into a reviewable plan, or state the question that prevents one.**

github.com/mario7gs/dev-journey-agent